

Task 1: Write a C program to demonstrate dynamic memory allocation using malloc(), calloc(), and realloc(). Allocate memory, initialize and display the allocated values, resize the memory block, and release the allocated memory using free().

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *a;
    int *b;
    int *c;

    a = malloc(5 * sizeof(int));
    if (a == NULL) {
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        a[i] = i + 1;
    }

    b = calloc(5, sizeof(int));
    if (b == NULL) {
        free(a);
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        b[i] = (i + 1) * 10;
    }

    a = realloc(a, 10 * sizeof(int));
    if (a == NULL) {
        free(b);
        return 1;
    }

    for (int i = 5; i < 10; i++) {
        a[i] = i + 1;
    }

    c = malloc(100 * sizeof(int));
    if (c == NULL) {
        free(a);
        free(b);
        return 1;
    }

    printf("Memory allocated using malloc():\n");
    for (int i = 0; i < 10; i++) {
        printf("%d ", a[i]);
    }

    printf("\n\nMemory allocated using calloc():\n");
    for (int i = 0; i < 5; i++) {
        printf("%d ", b[i]);
    }

    printf("\n\nMemory allocation using realloc() completed.\n");

    free(a);
    free(b);

    return 0;
}
```

Output:

```
karthik@Warthikeya:~$ mkdir -p ~/memory_allocation
cd ~/memory_allocation
karthik@Warthikeya:~/memory_allocation$ nano memory.c
karthik@Warthikeya:~/memory_allocation$ gcc -g memory.c -o memory
karthik@Warthikeya:~/memory_allocation$ ./memory
Memory allocated using malloc():
1 2 3 4 5 6 7 8 9 10

Memory allocated using calloc():
10 20 30 40 50

Memory allocation using realloc() completed.
karthik@Warthikeya:~/memory_allocation$ |
```

```

karthik@Warthikeya:~/memory_allocation$ valgrind --leak-check=full ./memory
==755== Memcheck, a memory error detector
==755== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==755== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==755== Command: ./memory
==755==
Memory allocated using malloc():
1 2 3 4 5 6 7 8 9 10

Memory allocated using calloc():
10 20 30 40 50

Memory allocation using realloc() completed.
==755==
==755== HEAP SUMMARY:
==755==    in use at exit: 400 bytes in 1 blocks
==755==    total heap usage: 5 allocs, 4 frees, 1,504 bytes allocated
==755==
==755== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==755==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==755==    by 0x109315: main (memory.c:41)
==755==
==755== LEAK SUMMARY:
==755==    definitely lost: 400 bytes in 1 blocks
==755==    indirectly lost: 0 bytes in 0 blocks
==755==    possibly lost: 0 bytes in 0 blocks
==755==    still reachable: 0 bytes in 0 blocks
==755==    suppressed: 0 bytes in 0 blocks
==755==
==755== For lists of detected and suppressed errors, rerun with: -s
==755== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
karthik@Warthikeya:~/memory_allocation$

```

Task 2: Write a C program to demonstrate the copy-on-write mechanism using fork(). Allocate memory dynamically, display the memory usage of the parent and child processes before and after modification, and observe how memory changes are handled independently after the fork.

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (1024 * 1024)

void show_memory(const char *name) {
    char command[256];

    snprintf(command, sizeof(command),
             "grep -E '^(Rss|Pss|Private|Shared)' /proc/%d/smmaps_rollup",
             getpid());

    printf("\n%s - PID: %d\n", name, getpid());
    fflush(stdout);

    system(command);
}

int main() {
    int *data = malloc(SIZE * sizeof(int));

    if (data == NULL) {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < SIZE; i++) {
        data[i] = 1;
    }

    printf("Parent before fork:\n");
    show_memory("Parent");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0) {
        printf("\nChild after fork, before modification:\n");
        show_memory("Child");

        printf("\nChild modifying data...\n");

        for (int i = 0; i < SIZE; i++) {
            data[i] = 2;
        }

        printf("\nChild after modification:\n");
        show_memory("Child");

        free(data);
        exit(0);
    } else {
        wait(NULL);

        printf("\nParent after child finishes:\n");
        show_memory("Parent");

        free(data);
    }

    return 0;
}

```

Output:

```
karthik@Warthikeya:~/memory_allocation$ mkdir -p ~/copy_on_write
cd ~/copy_on_write
karthik@Warthikeya:~/copy_on_write$ nano cow.c
karthik@Warthikeya:~/copy_on_write$ gcc -O0 -g cow.c -o cow
karthik@Warthikeya:~/copy_on_write$ ./cow
Parent before fork:

Parent - PID: 780
Rss:          5836 kB
Pss:          4289 kB
Pss_Dirty:    4208 kB
Pss_Anon:     4196 kB
Pss_File:     93 kB
Pss_Shmem:    0 kB
Shared_Clean: 1628 kB
Shared_Dirty: 0 kB
Private_Clean: 0 kB
Private_Dirty: 4208 kB
Shared_Hugetlb: 0 kB
Private_Hugetlb: 0 kB

Child after fork, before modification:

Child - PID: 783
Rss:          5164 kB
Pss:          2165 kB
Pss_Dirty:    2120 kB
Pss_Anon:     2116 kB
Pss_File:     49 kB
Pss_Shmem:    0 kB
Shared_Clean: 960 kB
Shared_Dirty: 4168 kB
Private_Clean: 0 kB
Private_Dirty: 36 kB
Shared_Hugetlb: 0 kB
Private_Hugetlb: 0 kB

Child modifying data...

Child after modification:

Child - PID: 783
Rss:          5164 kB
Pss:          4215 kB
Pss_Dirty:    4170 kB
Pss_Anon:     4166 kB
Pss_File:     49 kB
Pss_Shmem:    0 kB
Shared_Clean: 960 kB
Shared_Dirty: 68 kB
Private_Clean: 0 kB
Private_Dirty: 4136 kB
Shared_Hugetlb: 0 kB
Private_Hugetlb: 0 kB

Parent after child finishes:

Parent - PID: 780
Rss:          5836 kB
Pss:          4289 kB
Pss_Dirty:    4208 kB
Pss_Anon:     4196 kB
Pss_File:     93 kB
Pss_Shmem:    0 kB
Shared_Clean: 1628 kB
Shared_Dirty: 0 kB
Private_Clean: 0 kB
Private_Dirty: 4208 kB
Shared_Hugetlb: 0 kB
Private_Hugetlb: 0 kB
karthik@Warthikeya:~/copy_on_write$ |
```